

Vision Day3 HW1 레포트

로봇 예비단원 주호진

1. 머신러닝(Machine Learning)

머신러닝은 인공지능의 한 분야로, 어떠한 문제에 대해 정해진 규칙으로 판단하는 것이 아니라 판단조건을 기계 스스로 학습하고 판단하는 경우를 머신러닝이라고 한다.

아래에 간단한 문제에 대해 최적의 선택을 한다고 가정해보겠다.

학교에서 집까지 가는 최적의 길을 고르시오.

	A 길	B 길
걸리는 시간	학교 -> 집 : 5 분	학교 -> 집 : 10 분

이러한 쉽고 간단한 문제의 답은 직관적으로 B 라고 찾아낼 수 있다.

해당 문제에 대한 조건은 걸리는 시간이라는 1 개의 조건만 있기 때문이다.

그러면 조건이 많은 경우의 문제에 대해 최적의 선택을 해본다고 가정해보겠다.

주어진 조건을 고려하여 최적의 제품을 선택하시오.

	A 제품	B 제품	C 제품	D 제품
브랜드	S 사	K 사	G 사	R 사
가격	399,000	279,000	540,000	189,900
디자인	일반 마감	일반 마감	프리미엄 마감	보급형 마감
속도	빠름	보통	매우 빠름	빠름

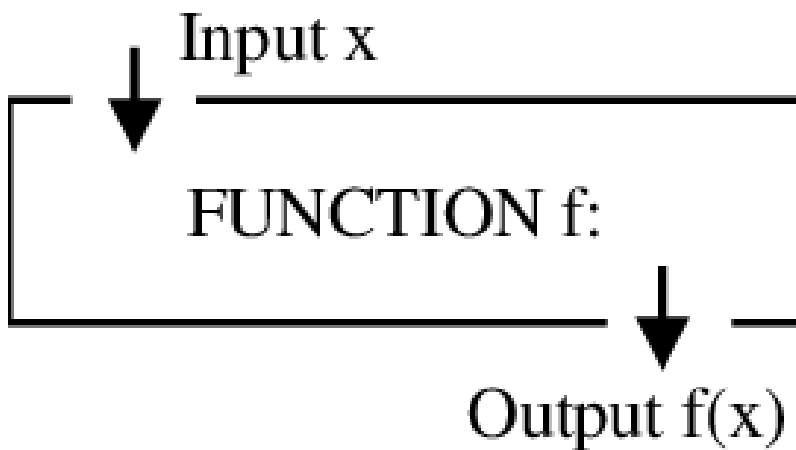
무게	보통	매우 가벼움	매우 무거움	무거움
----	----	--------	--------	-----

이런 식으로 고려해야 할 조건이 많아질수록 직관적으로 최적의 답을 고르지 못하게 된다.

여기에 현재 예산 상황이라든지, 해당 제품을 놓을 수 있는 공간적 제약이라든지, 제품을 구매할 수 있는 시간의 제약이라든지... 등의 고려해야 할 조건이 늘어날수록 경우의 수는 기하급수적으로 커지게 된다.

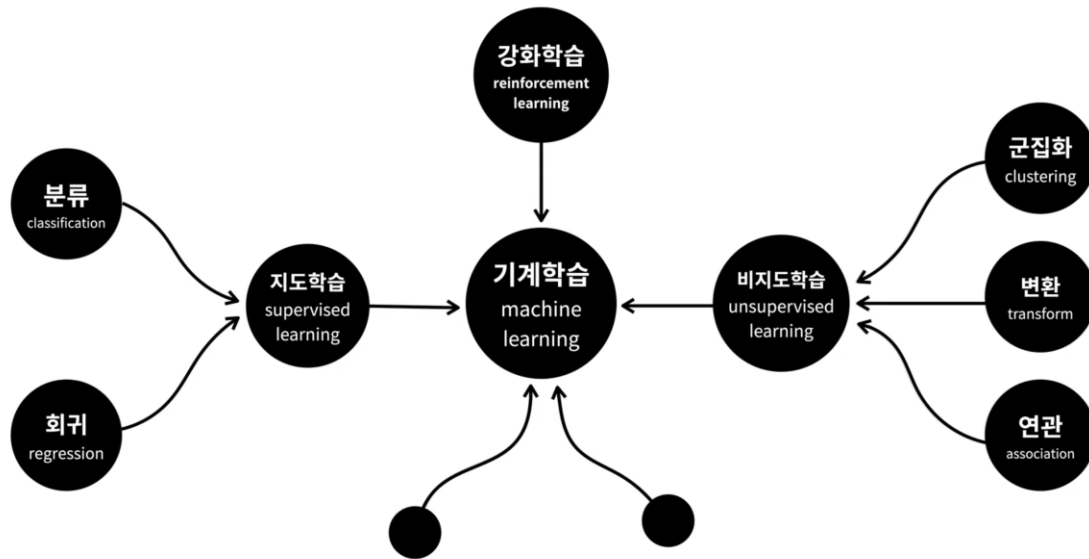
이렇게 복잡한 문제가 현실에는 차고 넘친다.

기존에는 복잡한 문제들을 사람이 판별했다면, 기술의 발전에 따라 사람이 판별하지 않고(AI), 기계가 직접 판별(머신러닝)하도록 한 것이다.



우리가 어렸을 적 수학에서 함수라는 개념을 넣을 때, 다음과 같은 사진으로 개념을 이해했다. 어떤 Input(x)을 넣으면 함수를 통해 Output($f(x) = y$)이 나온다.

위에서 예를 들었던 문제와 표가 Input 이 되고 어떤 문제의 최적인 결정이 Output 이라면 Input 에서 Output 을 도출해내는 Function 을 기존에 사람이 이러한 복잡한 조건들을 이리저리 조합해서 만들었다면, 기계학습을 통해 자동으로 function 을 만들어주는 편리한 도구로서 사용하게 되는 것이다.



머신 러닝은 크게 지도학습, 비지도학습, 강화학습으로 분류된다.

여기서 지도학습은 분류와 회귀, 비지도 학습은 군집화, 변환, 연관으로 나누어진다.

아래 설명에서 이 개념들을 다루어보도록 하겠다.

1-1. 지도학습(Supervised Learning)

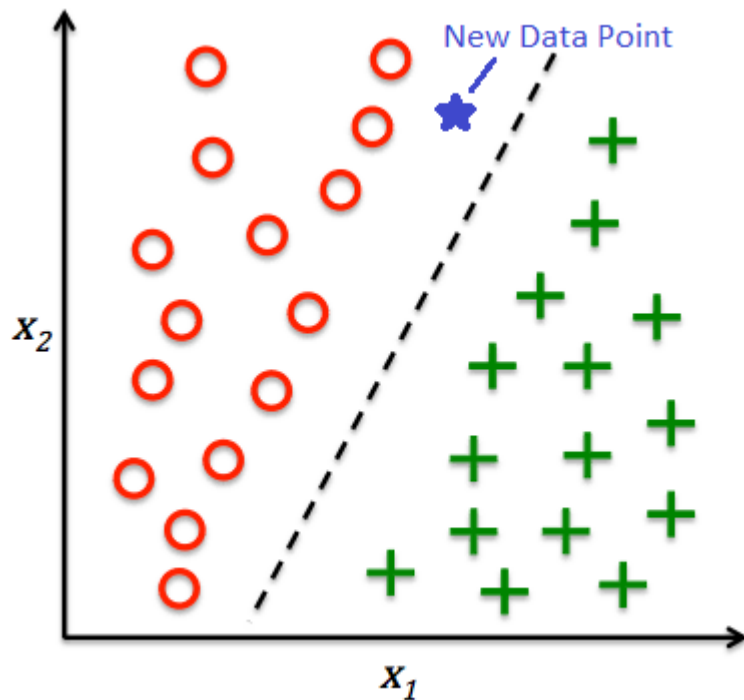
가장 먼저 지도학습이다.

지도학습은 정답이 포함된 데이터 셋을 사용해 입력과 출력 간의 관계를 학습하는 방식이다.

지도학습은 귀납적 추론과 비슷한데, 과거(데이터)가 이러했으므로 현재(input)도 이러할 것이다. 라는 추론을 하는 것이다.

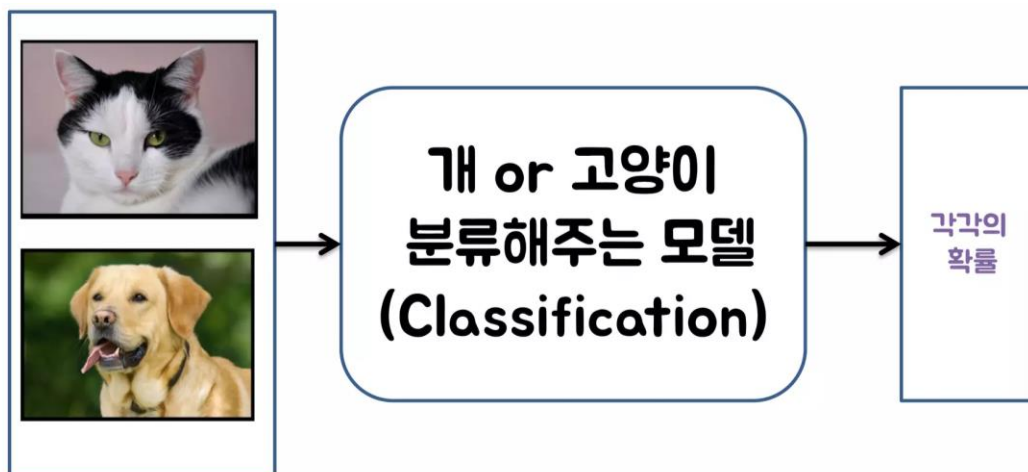
지도학습은 크게 분류와 회귀로 나뉜다.

1-1-1. 분류(Classification)



분류는 입력된 데이터를 사전에 정의된 카테고리로 나누는 것이다.

가장 많이 들게 되는 예시가 고양이인지 강아지인지 판별하는 분류이다.

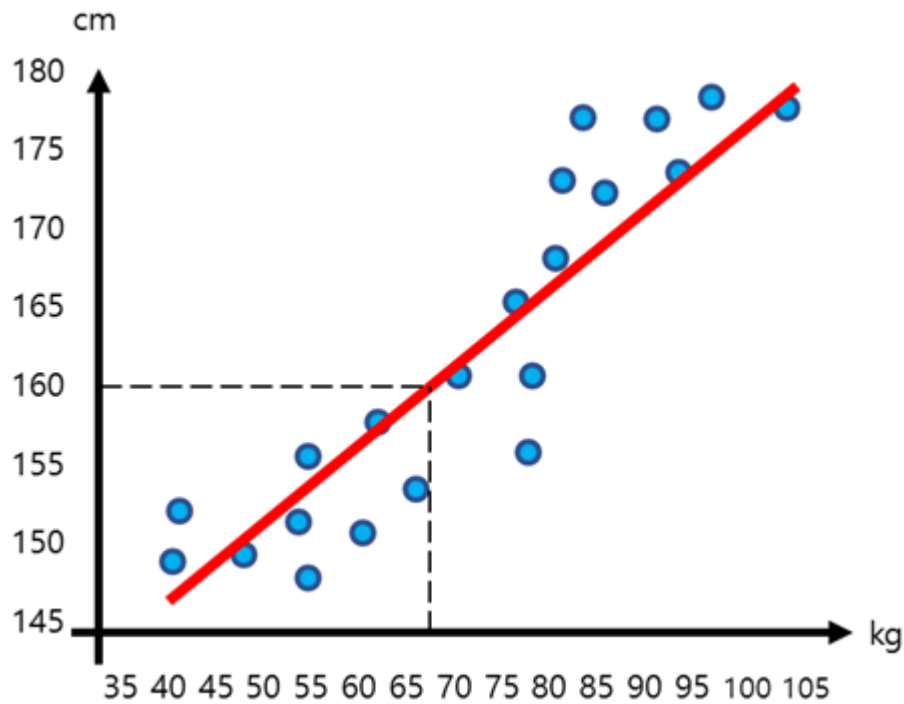


해당 예에서는 강아지, 아니면 고양이라는 2 개의 카테고리만 두고 입력 사진에 대해 강아지인지 고양이인지 어느쪽 확률이 더 높은지 판별하여 강아지, 또는 고양이라는 output 을 낸다.

우리가 흔히 쓰는 스팸 차단 기능도 이 분류 유형 중 하나라고 볼 수 있다. (스팸 / 스팸 아님)

1-1-2. 회귀(Regression)

회귀는 입력 데이터와 연관된 연속적인 값을 예측하는 것이다.



위 사진을 예시로 키와 몸무게가 매칭된 데이터셋을 받았다고 해보자.

이 데이터 셋을 통해 합리적으로 추론을 한다면 만약 키가 160 인 사람의 몸무게를 예측해본다고 했을 때 60~70kg 정도일 것으로 추측할 수 있다.

왜? -> 과거의 데이터가 키와 몸무게가 비례했기 때문이다.

만약 output 이 연속적인 몸무게 값이 아닌 50, 70, 90kg 처럼 유한 개의 카테고리가 지정된다면 이 문제는 다시 분류 문제로 볼 수 있다.

이처럼 분류와 회귀의 차이는 출력(output)이 연속적이냐(지정되어 있지 않은지), 연속적이지 않냐(지정되어 있는지)의 차이점이 존재하게 된다.

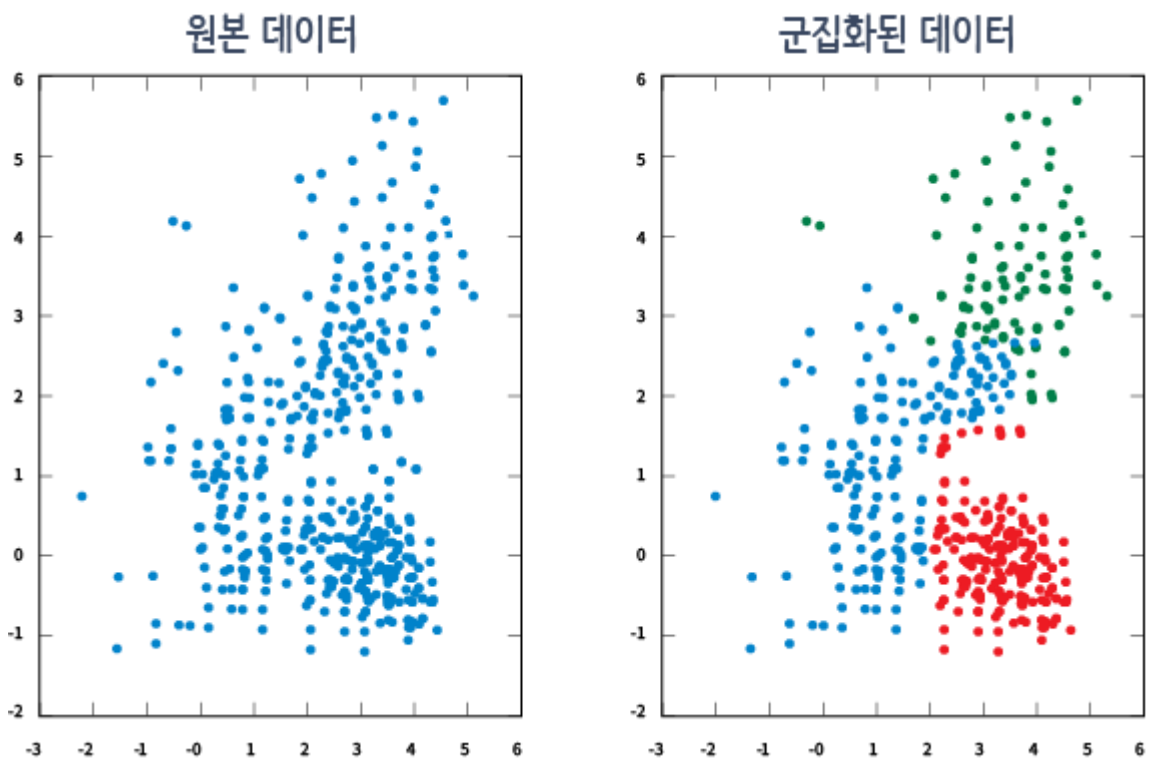
1-2. 비지도학습(Unsupervised Learning)

비지도 학습은 지도학습과 다르게 정답이 주어지지 않은, 입력 데이터만을 사용해 데이터의 관계를 찾아내는 학습 방식이다.

비지도 학습은 크게 군집화, 변환, 연관으로 나뉜다.

1-2-1. 군집화(Clustering)

유사한 특성을 가진 데이터들을 그룹으로 묶는 것을 군집화라고 한다.



지도학습의 분류와 다소 비슷해보일 수 있는데, 군집화는 있는 데이터들의 그룹을 묶어주는 과정이고, 분류는 그룹으로 나뉜 데이터셋을 통해 학습하여 어떤 값(Input)이 들어왔을 때 해당 값이 어떤 그룹인지 판단하는 점에서 차이가 있다.

1-2-2. 변환(Transform)

변환은 정답이 없는 원본 데이터를 사람이나 다른 머신러닝 알고리즘이 쉽게 이해할 수 있도록 새로운 형태로 변환해주는 기법이다.

예를 들어 고해상도의 이미지의 경우 다른 머신러닝이 계산할 때 수월할 수 있도록 주요 핵심 내용은 유지하고 중요하지 않은 내용은 버려서 용량을 줄일 수 있다.

이처럼 사람이나 기계가 학습하기 전 전처리 역할로 사용되는 경우가 많다.

1-2-3. 연관(Association)

연관은 데이터 사이의 특징을 찾아내는 것이다.

장바구니에 대한 예시가 많은데 예를 들어, 빵을 사는 사람은 잼이나 버터, 우유를 구매할 가능성이 높으므로 빵과 잼, 빵과 버터, 빵과 우유 사이의 연관이 높다고 볼 수 있다.

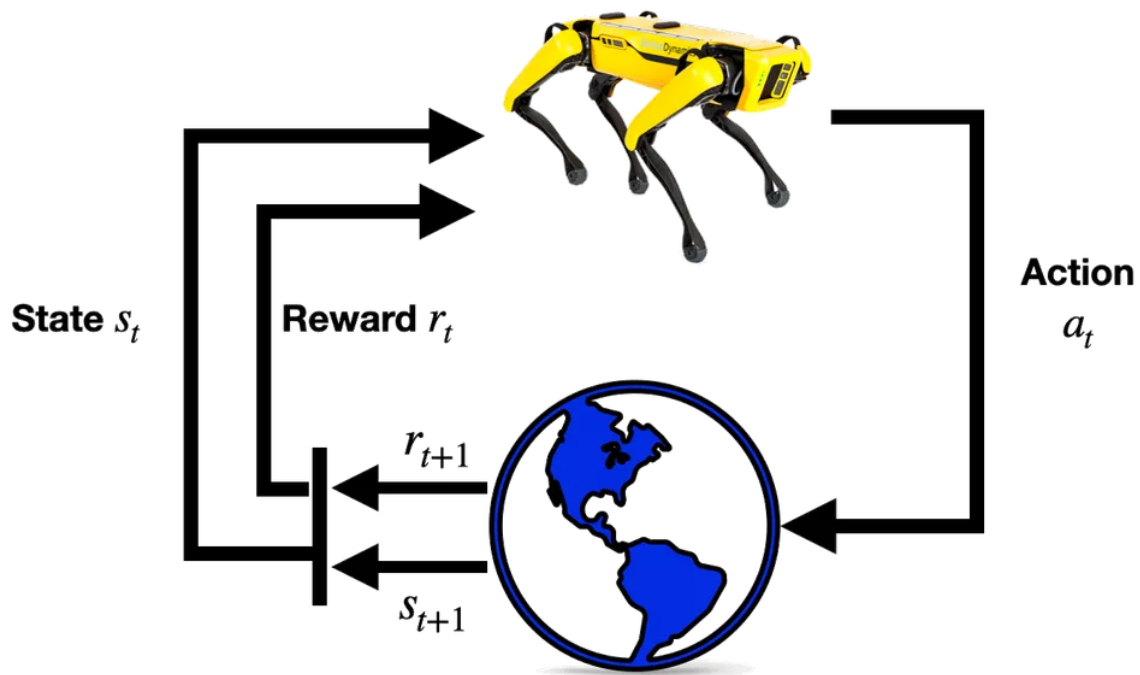
반대로 빵을 사는 사람이 갑자기 공구를 같이 살 확률은 낮고, 빵과 공구에 대한 어떤 접점이 적은 경우 빵과 공구 사이의 연관이 낮다고 볼 수 있다.

쿠팡과 같은 쇼핑 앱에서 예전에 샀던 제품이나, 최근에 봤던 제품과 비슷한 물품의 추천이 뜨는 경우 연관을 이용해 관련해서 구매율이 높은 물품을 추천하는 거라고도 볼 수 있다.

1-3. 강화학습(Reinforcement Learning)

강화학습은 에이전트(Agent)라는 학습 주체가 환경(environments)과 상호작용을 하며, 시행착오를 통해 학습하는 방식이다.

에이전트는 특정 상태(State)에서 행동(Action)을 하고 그 결과로 보상(Reward)을 받게 되는데, 이때 상점 또는 벌점을 받게 된다. 강화학습 알고리즘은 이 보상을 최대로 얻을 수 있는 방향으로 정책을 최적화하게 된다.



강화학습을 통한 보행 제어가 대표적 예시인데 보행의 경우 상태는 로봇 관절의 각도, 속도 등의 정보가 될 것이고, 행동은 해당 State(관절의 각도, 속도 등)을 얼마만큼, 어느 속도, 얼마만큼의 힘으로 움직이냐가 되고, 보상은 넘어지지 않고 잘 걸어다닌다면 상점, 넘어지거나 이상한 방식으로 보행하는 경우 벌점을 주게 되는 형식이 된다.

이러한 보상을 통해 어느 상황(State)에서 어떤 행동(Action)을 할 건지에 대한 정책(Policy)을 강화해나가는 것이다.

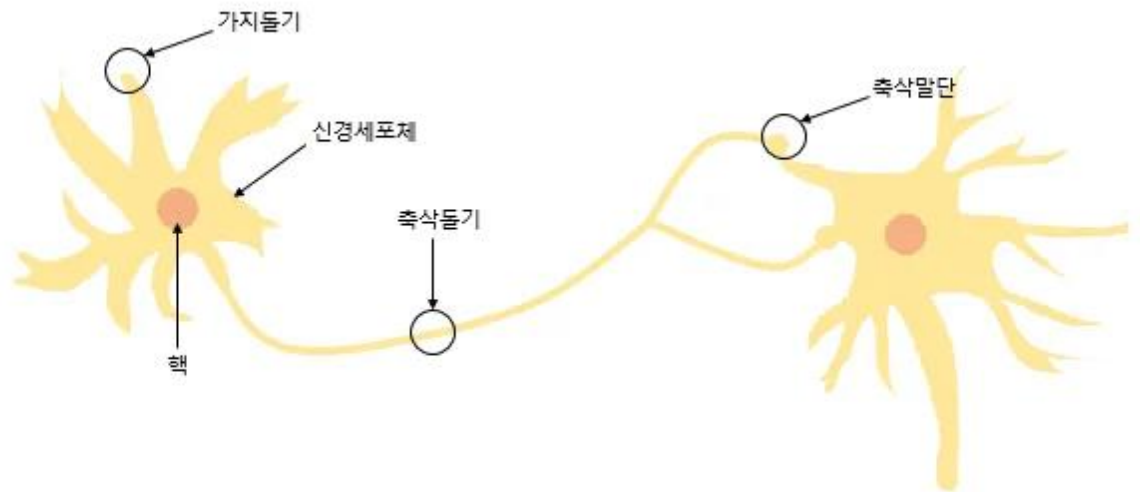
2. 딥러닝(Deep Learning)

딥러닝은 인공지능망을 기반으로 은닉층을 거쳐 특징을 스스로 학습하는 머신러닝의 한 분야이다.

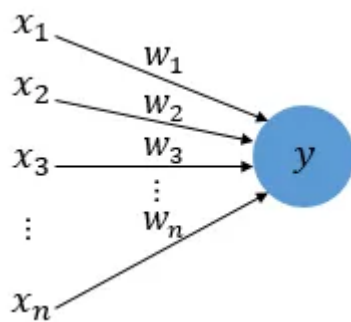
딥러닝에 대한 이론은 예전부터 있었으나, 인터넷이 보급되고 빅데이터가 퍼져나감과 동시에 GPU가 발전하게 되면서 사용하게 되었다.

2-1. 인공 신경망

인공 신경망은 퍼셉트론이라는 초기 형태의 이론에서 발전한 개념이다.



퍼셉트론은 인간의 뇌를 구성하는 뉴런의 구조를 따왔다. 뉴런은 아주 미세한 전기신호로 정보를 전달하는데 이를 모방하여 아래와 같은 구조를 취한다.



여러 개의 입력 신호를 받고, 각각의 입력값(x)에 가중치(w)를 곱해서 모두 더한 값을 받아 해당 값이 어떤 임계점을 넘으면 0 또는 1 을 출력하는 방식이다.

식으로 표현하면 $(w_1x_1 + w_2x_2 + \dots > \text{임계점}) \rightarrow 1$ 으로 표현할 수 있다.

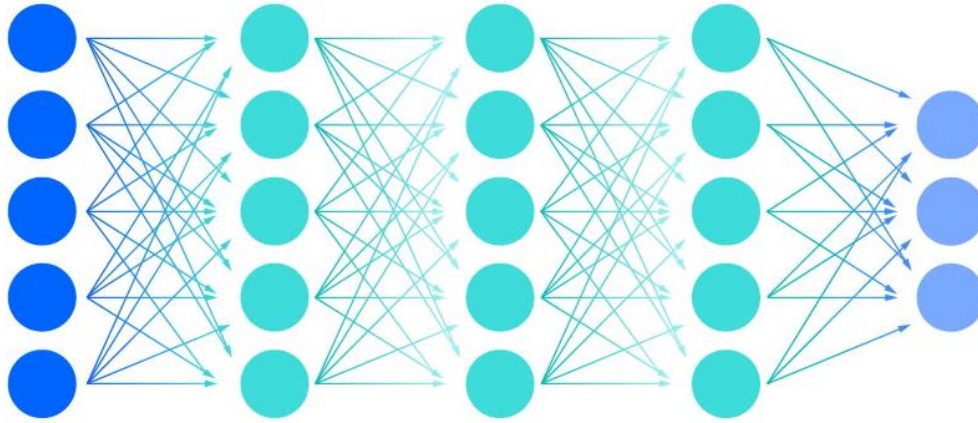
또는 임계점을 b 로 두고 이항하면 $(w_1x_1 + w_2x_2 + \dots - b > 0) \rightarrow 1$ 처럼 표현할 수도 있다.

이때 b 는 편향(bias)이라고 한다.

여기서 0 또는 1 을 출력하는 것을 뉴런, 또는 노드라고 부른다.

이러한 퍼셉트론 이론은 선형적인 분류는 할 수 있지만, 비선형적인 분류는 할 수 없다는 한계를 가지고 있다.

비선형적 분류를 할 수 있도록 퍼셉트론을 여러 층으로 쌓고 한 번이 아닌, 여러 번을 거치도록 만든 것이 인공 신경망이라고 할 수 있다.



2-2. 활성화 함수

위 사진처럼 여러 겹으로 인공신경망을 만들어도 결국 퍼셉트론 식에 따른 계산을 하다보면 결과값에서 입력이 엄청 많은 퍼셉트론 하나의 식으로 귀결되게 된다.

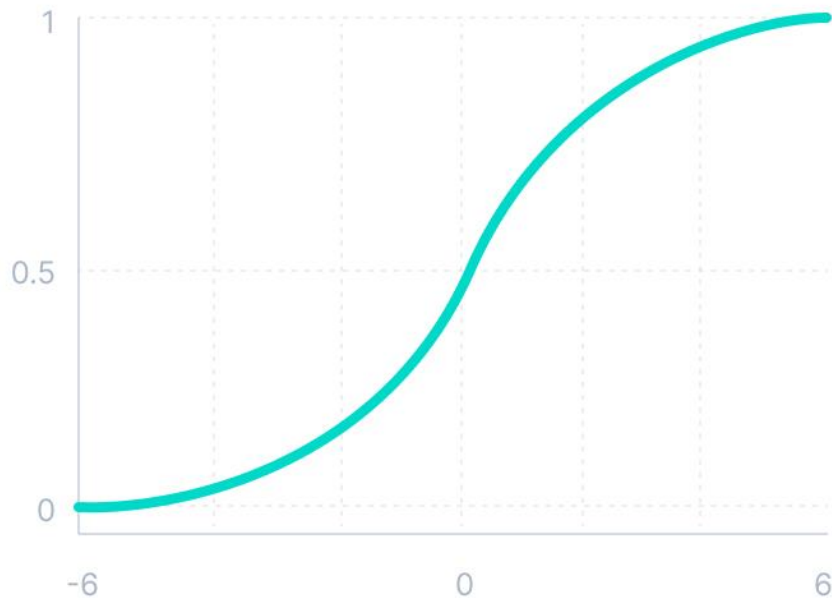
이 말은 단순히 여러 층으로 쌓기만 한다고 하면, 선형적 분류에서 벗어날 수 없다는 뜻이 된다.

따라서 비선형적 분류를 하기 위해 노드에 활성화 함수라는 걸 추가하게 된다.

노드에 들어온 $w_1x_1 + w_2x_2 + \dots$ 의 값을 활성화 함수라는 단계를 한번 더 거쳐서 이 데이터의 중요도를 판단하는 필터의 역할을 하게 되는 것이다.

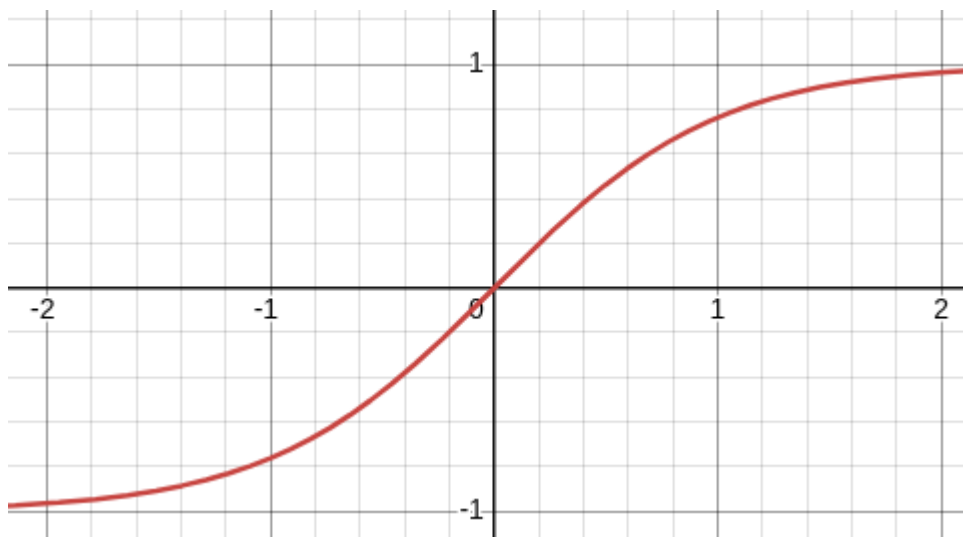
자주 쓰이는 활성화함수는 시그모이드, tanh, 렐루(ReLU) 등이 있다.

2-1-1. 시그모이드 함수



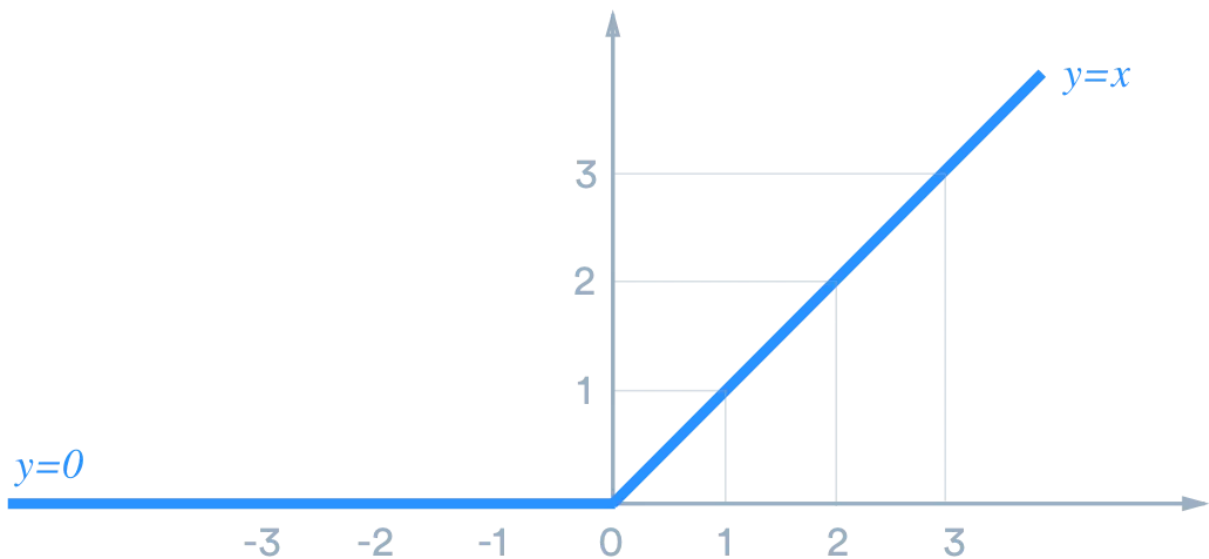
0~1 사이의 값을 갖는 함수이고, 확률로 해석하기에 직관적인 모델이다. 다만 입력값이 너무 크거나 작으면 미분값이 0에 가까워진다는 단점이 있다.

2-1-2. tanhx



시그모이드 함수 개형에서 원점을 지나고 음수 결과값을 가지게 되는 함수이다. 음의 결과값도 나올 수 있기 때문에 학습 속도가 빨라질 수 있는 장점이 있다. 시그모이드와 동일하게 값이 너무 크거나 작으면 미분값이 0으로 수렴하게 되는 단점이 있다.

2-1-3. ReLU



입력값이 0 보다 큰 경우에는 $y=x$ 꼴의 개형을 보이고 0 보다 작은 경우에는 모두 0 으로 취급하는 함수이다. 입력값이 양수라면 미분값이 항상 1 이 된다.

2-3. 심층 신경망(DNN: Deep Neural Network)

인공 신경망 중에서 입력층과 출력층을 제외한 층을 은닉층이라고 하는데 이 은닉층이 2 개 이상인 경우를 심층 신경망이라고 한다.

여기서 은닉층이란 딥러닝의 핵심이 되는 공간으로, 입력 데이터의 특징을 추출하는 단계이다. 층이 깊을수록 복잡한 패턴을 학습할 수 있다.

사실 위에서 본 인공 신경망의 예시 사진은 은닉층이 3 개이므로 인공 신경망이자 심층 신경망이라고 할 수 있다.

심층 신경망은 이전 층의 노드와 다음 층의 노드가 모두 연결되어 있는 구조이다. 따라서 노드가 많아질 수록 연결된 선이 많고 가중치와 편향이 많아지기 때문에 효율적인 자원 사용을 위해 최적화 기법 사용이 동반되어야 한다.

심층 신경망은 층을 깊게, 많이 쌓을수록 모델이 특징을 추출할 수 있는 폭이 커져 표현력이 증가하지만 과적합이나 층을 여러번 거치면서 0 이나 1 에 값이 가까워져 특정 활성화함수에서 기울기 소실 등이 일어날 수도 있다.

2-4. 비용 함수와 경사 하강법

심층 신경망 구조에서 내놓은 예측값이 정답과 얼마나 유사한지(얼마나 차이가 나는지)를 알아야 스스로 학습할 수 있게 된다. 이때 비용 함수와 경사하강법이 사용된다.

2-4-1. 비용 함수

비용 함수는 위에서 말했지만 예측값과 정답이 얼마나 차이가 나는지를 나타내는 함수라고 보면 된다.

만약 어떤 예측 값이 7 이고 진짜 정답은 10 이면 이때 오차는 3 이고 비용 함수가 오차를 그대로 적용한다면 비용 함수의 값도 3 이 될 것이다.

그런데 그대로 적용하면 문제가 생긴다.

	첫번째	두번째	세번째	네번째
예측 값	7	13	8	12
정답	10	10	10	10
오차	3	-3	2	-2

위와 같은 예측 결과값이 있다고 쳤을 때, 분명 예측 값은 정답과 오차가 있는데 비용 함수가 오차를 그대로 적용하여서 오차를 전부 더하게 되면 0 이 나오게 된다.

이러면 실제로는 오차가 있는데도 불구하고, 비용 함수를 통한 오차가 0 이니 학습이 완벽하게 됐다고 착각하게 된다.

이를 방지하기 위해서 비용 함수에 다양한 기법을 쓰는데, 대표적으로 평균 제곱 오차가 있다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - t_i)^2$$

오차를 그냥 더하는 것이 아니라 제곱하여 더해준다. 위 표의 경우 MSE 를 통해 오차를 구하게 되면 6.5 라는 오차값이 나오게 된다.

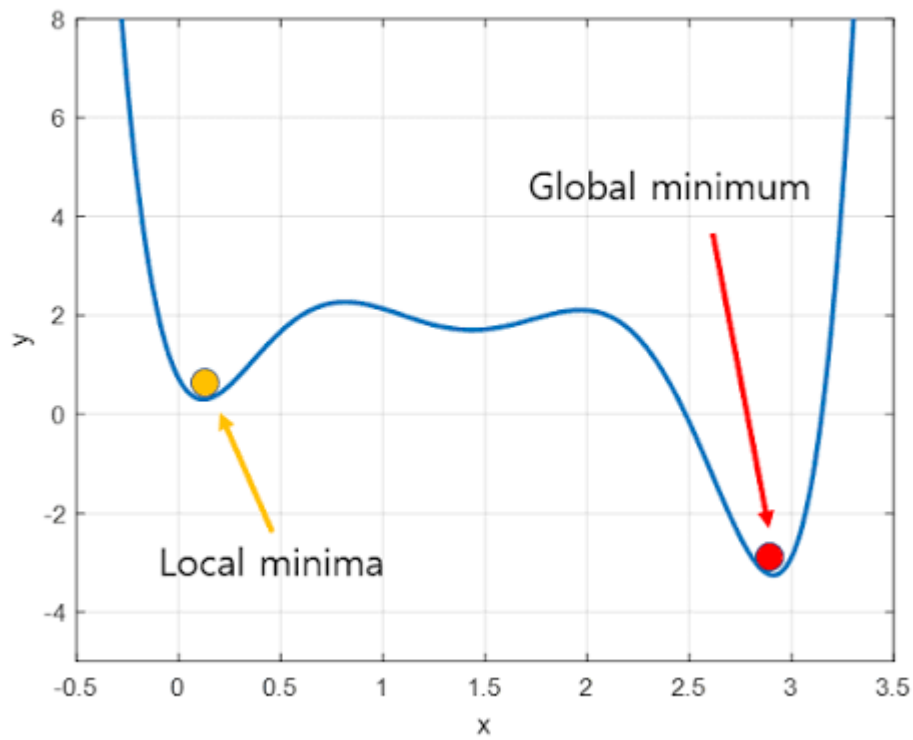
2-4-2. 경사 하강법

비용 함수로 오차를 구했다면, 오차를 최소화해야 한다. 오차를 최소화하기 위해 가중치와 편향을 조정해야 하는데, 이때 경사하강법이라는 것을 사용한다.

경사하강법은 말그대로 일계도함수(기울기)를 구해서 기울기가 줄어드는 방향으로 이동하게 하는 방법이다. 그래프 상에서 임계점을 찾아가는 거라고 볼 수 있다.

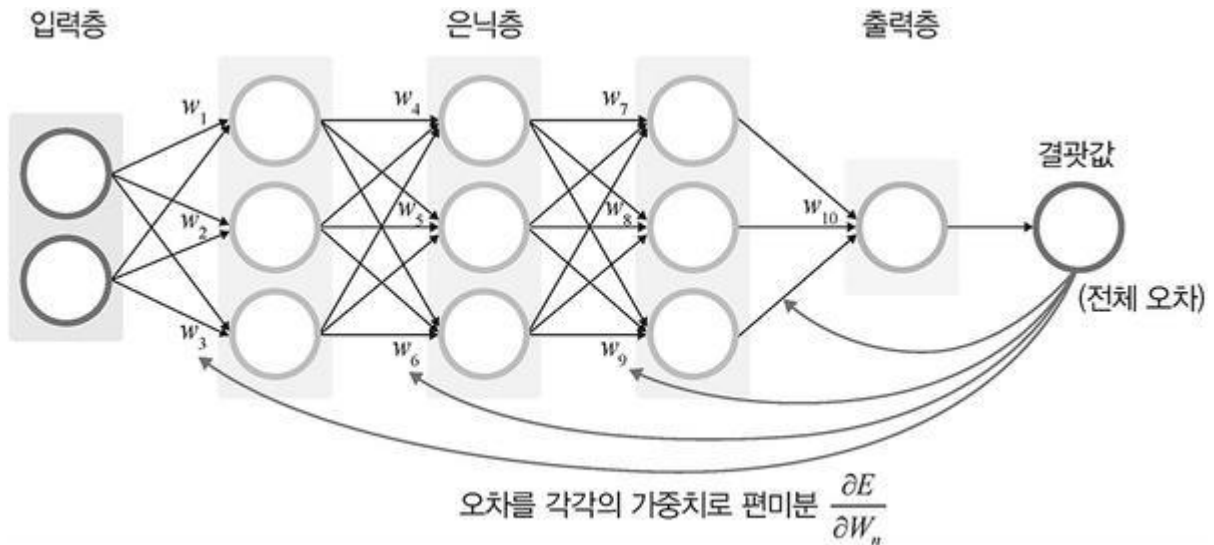
임계점을 찾아갈 때 현재 지점에서 구한 기울기 값에 학습률이라는 파라미터를 곱하여 움직이게 되는데, 학습률의 적절한 값을 설정해주어야 임계점을 찾는 시간이 줄어들게 된다.

학습률이 큰 경우 너무 많이 움직여서 그래프 상 임계점을 기준으로 좌우로 발산하게 될 수 있고, 학습률이 너무 작은 경우 달팽이처럼 움직여 최저점에 도달하는 데에 너무 오랜 시간이 걸릴수도 있다.



그래프가 2 차함수처럼 예쁘게 임계점이 하나인 경우 Global minimum 을 찾아갈 수 있겠지만 그래프가 복잡한 경우(사진보다도 더 복잡한 경우) Global minimum 을 찾는 게 어렵기 때문에 Local minima 를 찾는 것을 목표로 한다.

2-5. 역전파

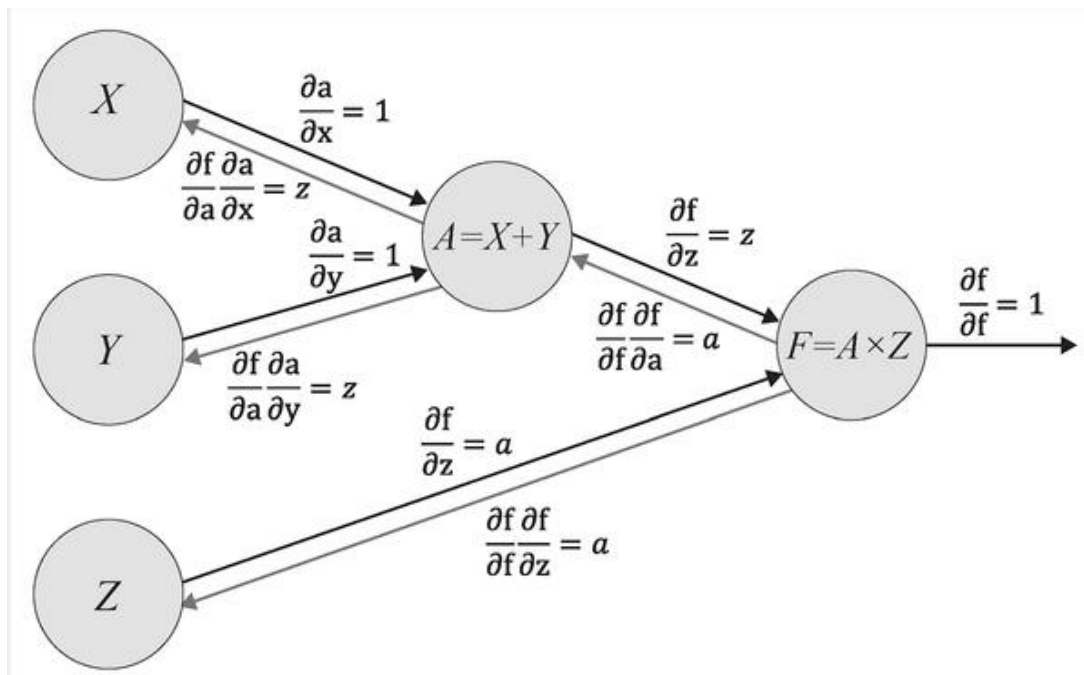


Copyright © Gilbut, Inc. All rights reserved.

신경망의 은닉층이 매우 깊어지면 각 층의 가중치가 최종 오차에 미치는 영향을 하나하나 구하기가 매우 어려워진다. 이런 점을 해결하기 위해 역전파라는 개념이 생겼다.

역전파는 출력층에서 발생한 최종 오차를 반대로 전파시키면서 각 층의 가중치와 편향을 한번에 업데이트 해주는 알고리즘이다.

미적분의 연쇄 법칙을 적용해서 전체 미분값이 각 개별들의 곱으로 이루어져있다는 개념을 역이용한다.



출력층이 가장 오른쪽에 존재한다고 봤을 때 F 노드에는 F에 대한 편미분, A 노드에선 a에 대한

편미분...을 해주게 되면 연쇄법칙에 의해 모든 계산이 한번에 진행된다.

3. PyTorch 라이브러리/클래스

퍼셉트론

Torch.nn.Linear 을 사용한다.

Nn 은 neural network 의 줄말이며, 앞서 설명했듯 퍼셉트론은 선형적 추론을 하는 역할이기 때문에 Linear 가 붙었다.

```
import torch
import torch.nn as nn

layer = nn.Linear(in_features=10, out_features=5) # 입력 10, 출력 5 개
input_data = torch.randn(3, 10)                 # (배치 크기 3, 입력 10)
output = layer(input_data)                        # 결과 크기: (3, 5)
```

활성 함수

torch.nn.ReLU, torch.sigmoid, torch.tanh 등 사용할 함수를 선언하여 사용한다.

비용 함수

활성함수와 비슷하게 torch.nn.MSELoss 와 같이 선언하여 사용한다.

```
import torch
import torch.nn as nn

criterion = nn.MSELoss()
prediction = torch.tensor([2.5, 0.0])
target = torch.tensor([3.0, 0.0])

loss = criterion(prediction, target) # MSE
print(loss.item())
```

경사 하강법

`torch.optim.SGD`, `torch.optim.Adam`, `torch.optim.RMSprop` 등 사용할 경사하강법을 고르고 학습률(lr)을 설정한다.

```
import torch.optim as optim

model = nn.Linear(5, 1)
optimizer = optim.Adam(model.parameters(), lr=0.01)
optimizer.step()
```

역전파

`loss.backward()`을 아래 순서와 같이 사용한다.

```
optimizer.zero_grad() # 1. 이전 기울기 초기화
loss.backward()       # 2. 역전파로 기울기 계산
optimizer.step()      # 3. 가중치 갱신
```

심층 신경망

`torch.nn.Sequential` 또는 `torch.nn.Module` 을 사용해서 심층 신경망 구성한다.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 16),
    nn.ReLU(),
    nn.Linear(16, 1)
)

output = model(torch.randn(3, 10))
```